

AWS Breach Simulation: Multi-Attack Incident Response Report

Author: Paul "ZETEO" Ayegbusi
Date: May 23, 2026
Classification: Training Exercise

Executive Summary

This report documents the detection, investigation, and analysis of three simulated cloud attacks executed against an AWS environment using Stratus Red Team. The attacks targeted credential access, defense evasion, and persistence techniques commonly observed in real-world cloud breaches.

Key Findings

- All three attack techniques were successfully recorded in AWS CloudTrail with detailed forensic evidence.
- AWS GuardDuty did not generate automated findings for any of the simulated attacks, exposing notable detection gaps.
- Manual CloudTrail analysis was required to identify and investigate malicious activity.
- Attackers could potentially maintain undetected access if organizations rely solely on automated security tooling.

Impact Assessment

Attack	Impact
Credential Access	30 unauthorized attempts to retrieve EC2 Windows password data were blocked by IAM permissions
Defense Evasion	CloudTrail logging was successfully disabled on a secondary trail
Persistence	A privileged IAM administrator user was created, establishing persistent access

Recommendation

Organizations should implement layered detection strategies including CloudWatch alarms, SIEM correlation rules, and proactive threat hunting procedures to supplement native cloud security tooling.

Attack Timeline

All timestamps are in UTC.

Time (UTC)	Event	Attack Technique	MITRE ATT&CK
01:41:52	Attacker executed <code>stratus detonate aws.credential-access.ec2-get-password-data</code>	Credential Access	T1552.005
01:42:00	30 <code>GetPasswordData</code> API calls logged in CloudTrail	EC2 password retrieval attempts	T1552.005
01:42:00	All requests returned <code>Client.UnauthorizedOperation</code>	Attack blocked by IAM permissions	-
04:33:22	Attacker executed <code>stratus detonate aws.defense-evasion.cloudtrail-stop</code>	Defense Evasion	T1562.008
04:33:22	<code>StopLogging</code> API call disabled CloudTrail logging	Logging disabled on test trail	T1562.008
09:42:15	Attacker executed <code>stratus detonate aws.persistence.iam-create-admin-user</code>	Persistence	T1136.003
09:42:15	<code>CreateUser</code> and <code>AttachUserPolicy</code> API calls created an admin IAM user	Persistent administrative access established	T1136.003

Note: GuardDuty generated zero findings across all attack phases despite CloudTrail successfully logging the malicious activity.

Technical Analysis

Attack1:EC2PasswordData Retrieval

Attack Description

The attacker simulated credential theft by attempting to retrieve administrator password data from Windows EC2 instances using the `ec2:GetPasswordData` API. This technique is commonly used after AWS credential compromise to escalate access into compute resources.

Execution Method

The simulation targeted 30 randomly generated EC2 instance IDs to increase the likelihood of identifying a valid Windows instance with retrievable credentials.

CloudTrail Evidence

Field	Value
Event Name	GetPasswordData
User Identity	aws-go-sdk-1779500511003765619
Source IP	143.105.174.63
Address	
AWS Region	us-east-1
Result	Client.UnauthorizedOperation

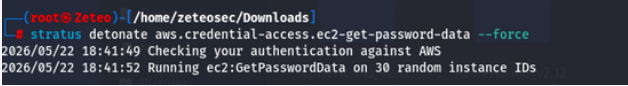
Root Cause

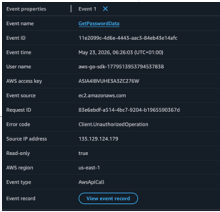
The IAM principal used by Stratus Red Team lacked the `ec2:GetPasswordData` permission. Although unsuccessful, the repeated failed API calls demonstrated reconnaissance and privilege probing behavior.

MITRE ATT&CK Mapping

Category	Technique
Tactic	Credential Access
Technique	Unsecured Credentials: Cloud Instance Metadata API (T1552.005)

Screenshots





Attack 2: CloudTrail Logging Disabled

Attack Description

The attacker disabled CloudTrail logging to evade detection and conceal subsequent malicious activity. Disabling audit logging is a common anti-forensics technique used during cloud intrusions.

Execution Method

The attack targeted the Stratus-created trail:

```
stratus-red-team-ct-stop-trail-gbphdsorpp
```

The attacker successfully invoked the `cloudtrail:StopLogging` API.

CloudTrail Evidence

Field	Value
Event Name	StopLogging
Trail Name	stratus-red-team-ct-stop-trail-gbphdsorpp
Source IP Address	135.129.124.179
Result	Success

User Identity

```
{
  "type": "IAMUser",
  "principalId": "AIDA4IBVUHE3BBISVUBXS",
  "arn": "arn:aws:iam::841926064438:user/Zeteo",
  "accountId": "841926064438",
  "accessKeyId": "AKIA4IBVUHE3IWBSWSMV",
  "userName": "Zeteo"
}
```

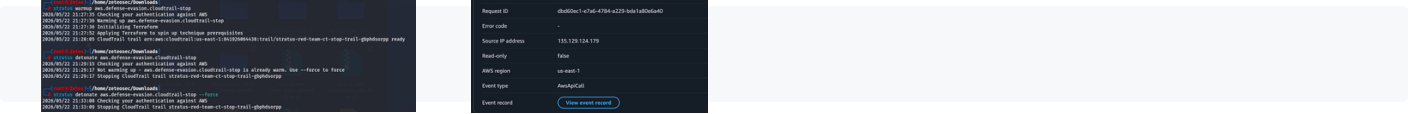
Root Cause

The IAM principal possessed excessive CloudTrail management permissions, allowing the attacker to disable logging without restriction.

MITRE ATT&CK Mapping

Category	Technique
Tactic	Defense Evasion
Technique	Impair Defenses: Disable Cloud Logs (T1562.008)

Screenshots



Attack 3: Backdoor IAM Admin User Creation

Attack Description

The attacker created a new IAM user with administrative privileges to establish persistent access within the AWS environment.

Execution Method

The attack used the following APIs:

- iam:CreateUser
- iam:AttachUserPolicy

The managed policy `AdministratorAccess` was attached to the newly created IAM user.

CloudTrail Evidence

Field	Value
Event Name	CreateUser, AttachUserPolicy
User Created	malicious-iam-user
Policy Attached	AdministratorAccess
Source iP Address	143.105.174.31
Result	Success

User Identity

```
{
  "type": "IAMUser",
  "principalId": "AIDA4IBVUHE3BBISVUBXS",
  "arn": "arn:aws:iam::841926064438:user/Zeteo",
  "accountId": "841926064438",
  "accessKeyId": "AKIA4IBVUHE3IWBSWSMV",
  "userName": "Zeteo"
}
```

Root Cause

The compromised IAM credentials had excessive iam:* permissions, allowing unrestricted user and policy management actions.

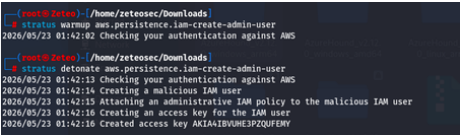
MITRE ATT&CK Mapping

Category	Technique
Tactic	Persistence
Technique	Create Account: Cloud Account (T1136.003)

Event properties | Event 1 X

Event name	CreateUser
Event ID	32f90723-23b4-4e9f-8c8f-ed822d870fc4
Event time	May 23, 2026, 09:42:15 (UTC+01:00)
User name	Zeteo
AWS access key	AKIA4IBVUHE3IWBSWSMV
Event source	iam.amazonaws.com
Request ID	f923a35f-af87-4d5d-a574-ea1cffd60364
Error code	-
Source IP address	143.105.174.31
Read-only	false
AWS region	us-east-1
Event type	AwsApiCall
Event record	View event record

Screenshots



Detection Gap Analysis

Why GuardDuty Failed to Alert

Despite GuardDuty being enabled and configured to monitor CloudTrail, VPC Flow Logs, and DNS logs, no findings were generated during any stage of the simulation.

Identified Causes

- Failed API Calls Were Not Prioritized
The GetPasswordData attempts failed due to insufficient permissions. GuardDuty prioritizes successful malicious activity over failed reconnaissance attempts.
- Test Trail Targeting
The simulation disabled a temporary Stratus-created CloudTrail trail rather than a production logging trail, reducing the perceived severity of the activity.
- No Established Behavioral Baseline
The AWS account was newly provisioned with limited historical activity. GuardDuty anomaly detection models rely heavily on established baselines.
- Finding Publication Delay
GuardDuty was initially configured with a 6-hour finding publication interval before being reduced to 15 minutes. Certain low-confidence detections may not surface immediately or at all.

5. Security Testing Environment Indicators

The use of known security testing frameworks such as Stratus Red Team may reduce alert confidence in isolated lab environments.

Real-World Implications

This exercise demonstrates that automated detection tooling alone is insufficient for comprehensive cloud threat detection.

Potential Risks

- A malicious IAM administrator user could persist undetected.
- CloudTrail logging could be disabled without immediate notification.
- Security teams relying solely on GuardDuty may lose visibility into active compromise activity.

Conclusion

Organizations should combine native cloud security tooling with SIEM correlation, custom detections, and continuous threat hunting processes.

Remediation Steps

Immediate Response Actions

Attack 1 — GetPasswordData Attempts

- No remediation required because IAM permissions blocked the requests.
- Review IAM policies to ensure `ec2:GetPasswordData` access is restricted.
- Validate that source IP `143.105.174.63` belongs to authorized testing activity.

Attack 2 — CloudTrail Disabled

- Re-enable CloudTrail logging immediately.
- Audit all IAM principals with `cloudtrail:StopLogging` permissions.
- Rotate credentials associated with CloudTrail management access.

Attack 3 — Backdoor IAM User

- Delete the unauthorized IAM user immediately.
- Revoke all associated credentials and console access.
- Review CloudTrail activity performed by the malicious user.
- Rotate credentials for the compromised IAM principal.

Detection Improvements

1. Custom CloudWatch Alarms

Unauthorized GetPasswordData Attempts

```
aws cloudwatch put-metric-alarm \  
  --alarm-name "Unauthorized-GetPasswordData-Attempts" \  
  --alarm-description "Alert on EC2 password retrieval attempts" \  
  --metric-name "GetPasswordData" \  
  --namespace "CloudTrailMetrics" \  
  --statistic "Sum" \  
  --period 300 \  
  --threshold 1 \  
  --comparison-operator "GreaterThanThreshold"
```

CloudTrail Logging Disabled

```
aws cloudwatch put-metric-alarm \  
  --alarm-name "CloudTrail-Logging-Disabled" \  
  --alarm-description "Alert when CloudTrail logging is stopped" \  
  --metric-name "StopLogging" \  
  --namespace "CloudTrailMetrics" \  
  --statistic "Sum" \  
  --period 60 \  
  --threshold 1 \  
  --comparison-operator "GreaterThanThreshold"
```

IAM Administrator User Creation

```
aws cloudwatch put-metric-alarm \  
  --alarm-name "Unauthorized-Admin-User-Creation" \  
  --alarm-description "Alert on new IAM users with admin policies" \  
  --metric-name "AdminUserCreation" \  
  --namespace "CloudTrailMetrics" \  
  --statistic "Sum" \  
  --period 300 \  
  --threshold 1 \  
  --comparison-operator "GreaterThanThreshold"
```

2. SIEM Correlation Rules

Microsoft Sentinel (KQL)

```
AWSCloudTrail  
| where EventName in ("GetPasswordData", "StopLogging", "CreateUser", "AttachUserPolicy")  
| where ErrorCode != "Client.UnauthorizedOperation"  
| summarize Count=count() by UserIdentityArn, SourceIPAddress, EventName, bin(TimeGenerated, 5m)  
| where Count > 5
```

Splunk (SPL)

```
index=cloudtrail (eventName="GetPasswordData" OR eventName="StopLogging" OR eventName="CreateUser")  
| stats count by userIdentity.arn, sourceIPAddress, eventName  
| where count > 5
```

3. Preventive Controls

Enforce IMDSv2 on EC2 Instances

```
aws ec2 modify-instance-metadata-options \  
  --instance-id <instance-id> \  
  --http-tokens required \  
  --http-put-response-hop-limit 1
```

Enable CloudTrail Log Immutability

```
aws s3api put-object-lock-configuration \  
  --bucket cloudtrail-logs-zeteo-1779498681 \  
  --object-lock-configuration '{"ObjectLockEnabled":"Enabled","Rule":{"DefaultRetention":{"Mode":"GOVERNANCE","Days":90}}}'
```

Apply Least-Privilege IAM Controls

- Restrict cloudtrail:StopLogging access to break-glass administrator roles only.
 - Restrict iam:CreateUser permissions.
 - Require MFA for all privileged IAM operations.
-

4. Proactive Threat Hunting

Establish recurring threat hunting activities to identify suspicious activity that bypasses automated detection.

Weekly Hunt Scenarios

- Identify IAM users created outside approved maintenance windows.
 - Detect API activity originating from unusual geographic regions.
 - Review CloudTrail logging interruptions or gaps.
 - Investigate repeated failed authorization attempts for reconnaissance patterns.
-

Conclusion

This incident response simulation demonstrated the effectiveness of AWS CloudTrail as a forensic data source while exposing critical limitations in automated threat detection tools such as AWS GuardDuty.

Key Takeaways

1. CloudTrail is Essential for Investigation
All three attack techniques were comprehensively logged with timestamps, user identities, source IP addresses, and API parameters, providing complete forensic visibility into attacker activity.
2. Automated Detection Has Blind Spots
GuardDuty did not generate findings for credential theft attempts, CloudTrail tampering, or unauthorized administrative account creation. Organizations should not rely solely on automated alerting mechanisms.
3. Layered Defense is Critical
Effective cloud security requires multiple detection and response layers, including automated threat detection, real-time monitoring, SIEM correlation, and proactive threat hunting.
4. IAM Permissions Directly Impact Security Posture
Attack 1 was prevented through least-privilege IAM controls, while Attacks 2 and 3 succeeded due to overly permissive credentials. Proper IAM configuration remains a foundational security control.

Skills Demonstrated

- Execution and analysis of multi-stage cloud attack simulations
- AWS CloudTrail log analysis and forensic investigation
- MITRE ATT&CK framework mapping for cloud-based threats
- Identification of detection gaps and remediation planning
- Development of custom detection logic for SIEM platforms

Next Steps

- Implement recommended CloudWatch alarms and SIEM detection rules in a production-like environment
 - Conduct additional simulations to validate detection improvements
 - Expand testing to include data exfiltration and lateral movement scenarios
-

Appendix: References

- Stratus Red Team Documentation — <https://stratus-red-team.cloud/>
 - MITRE ATT&CK Cloud Matrix — <https://attack.mitre.org/matrices/enterprise/cloud/>
 - AWS CloudTrail User Guide — <https://docs.aws.amazon.com/cloudtrail/>
 - AWS GuardDuty Best Practices — <https://docs.aws.amazon.com/guardduty/>
-

Author Contact

Zeteo

- Website: <https://zeteosec.com>
- GitHub: <https://github.com/0xZeteo>
- LinkedIn: <https://linkedin.com/in/paulayegbusi>